

//String Handling

Access Functions

`str[index]`: Accesses the character at the specified position (no bounds checking).
`at(size_t pos)`: Accesses the character at the specified position with bounds checking.
`front()`: Returns the first character of the string.
`back()`: Returns the last character of the string.

Capacity Functions

`size()` / `length()`: Returns the number of characters in the string.
`max_size()`: Returns the maximum number of characters the string can hold.
`capacity()`: Returns the current allocated capacity of the string.
`empty()`: Checks whether the string is empty.
`resize(size_t n, char c)`: Resizes the string to n characters, filling new positions with c.

Modifiers

`clear()`: Removes all characters from the string.
`insert(size_t pos, const std::string str)`: Inserts a string at the specified position.
`erase(size_t pos, size_t len)`: Erases a portion of the string starting at pos for len characters.
`replace(size_t pos, size_t len, const std::string& str)`: Replaces a substring with another string.
`push_back(char c)`: Appends a character to the end of the string.
`pop_back()`: Removes the last character of the string.
`append(const std::string& str)`: Appends another string to the end of the current string.
`str.append(temp_str);`
`swap(std::string& str)`: Swaps the contents of two strings.

Operations

`compare(const std::string& str)`: Compares the string with another string lexicographically.
`substr(size_t pos, size_t len)`: Returns a substring starting from pos with length len.
`find(const std::string& str)`: Finds the first occurrence of a substring.
`rfind(const std::string& str)`: Finds the last occurrence of a substring.
`find_first_of(const std::string& str)`: Finds the first character from a set of characters.
`find_last_of(const std::string& str)`: Finds the last character from a set of characters.
`find_first_not_of(const std::string& str)`: Finds the first character not in the specified set.
`find_last_not_of(const std::string& str)`: Finds the last character not in the specified set.

Iterators

`begin()` / `end()`: Returns an iterator to the beginning/end of the string.
`rbegin()` / `rend()`: Returns a reverse iterator to the beginning/end of the string.

Input/Output

`getline(std::istream& is, std::string& str)`: Reads a line from input and stores it in str.

Conversion Functions

`stoi(const std::string& str)`: Converts a string to an integer (int).
`stol(const std::string& str)`: Converts a string to a long integer (long).
`stof(const std::string& str)`: Converts a string to a float (float).
`to_string(int val)`: Converts an integer to a string. (Overloads exist for other numeric types.)

1. String Manipulation Utilities (<algorithm> and <cctype>)

transform: Applies a function to each character (e.g., converting case).

reverse: Reverses the string in-place.

replace: Replaces all occurrences of a character.

6. Character Classification (<cctype>)

toupper(char c): Converts a character to uppercase.

tolower(char c): Converts a character to lowercase.

isdigit(char c): Checks if the character is a digit.

isspace(char c): Checks if the character is whitespace.

isalpha(str.at(i))

7. String Comparison Utilities

lexicographical_compare: Compares two strings lexicographically.

8. Custom Utility Functions

Trim leading/trailing spaces: Use custom logic with std::find_first_not_of and std::find_last_not_of.

```
///Make A Substring Of A String
```

```
//substr(starting index, number of elements)
```

```
temp = str.substr(0, 3);
```

```
///Insert A New_String Anywhere(Any Index) Inside Another String
```

```
str.insert(0, new_string);
```

```
///Search For The Index Position Of A Character Or A Substring Inside A String
```

```
int index = str.find(s);
```

```
///Erase Portion Of A String
```

```
//erase(starting index, number of character to erase);
```

```
str.erase(0, 4);
```

```
///To Swap String
```

```
str.swap(src);
```

```
///Reverse A String
```

```
reverse(str.begin(), str.end());
```

///Compare string with other string

```
///Replace A Portion Of A String With Another String
```

```
//replace(start index, number of char to replace, another string);
```

```
str.replace(i,1,s);
```

```
///Removes Last Char Of A String
```

```
str.pop_back();
```

```
///Add A Char(Not String) In The End Of A String
```

```
    str.push_back(ch);

    return 0;
}

//For strings like MmmMMEEeeeEEooOooOwwwWWWW
int main(){
    int t,n;
    string s;
    for(cin>>t;cin>>n>>s;)
        cout<<(regex_match(s,regex("[mM]+[eE]+[oO]+[wW]+"))?"YES":"NO")<<endl;
}
```

//Regex

5. Regular Expressions (<regex>)

regex_match: Matches a string against a regular expression.

regex_search: Searches for a regular expression match within a string.

regex_replace: Replaces parts of a string that match a regular expression.

//Replaces every string "hi" with the string "hello" on the whole string

```
string s;
cin >> s;
regex p("hi");
cout << regex_replace(s, p, "hello");

string s, ch, change;
cin >> s >> change;
cout << "With: ";
cin >> ch;
regex str(change);
change = regex_replace(s, str, ch);
cout << change << endl;
}
```

//Binary Search

```
int BinarySearch(int arr[], int target, int low, int high){
    int mid;
    while( low <= high ) {
        mid = low + ( high - low ) / 2;
        if( arr[mid] == target )
            return mid;

        if( arr[mid] < target )
            low = mid + 1;
        else if( arr[mid] > target )
            high = mid - 1;
    }
}
```

```
    return -1;  
}
```

//Set how many numbers after .

```
cout << fixed << setprecision(6) << double_seq << endl;
```

//Find Maximum Or Minimum Value In An Array

```
cout << "Max: " << *max_element(arr, arr + sizee);
```

```
cout << "Min: " << *min_element(arr, arr + sizee);
```

//Find The Index Of Maximum Or Minimum Value In An Array

```
cout << "Index: " << min_element(arr, arr + sizee) - arr;
```

```
cout << "Index: " << max_element(arr, arr + sizee) - arr;
```

//For Vector

```
int low = min_element( skill.begin(), skill.end() ) - skill.begin();
```

```
    cout << low << endl;
```

//Sort An Array

```
sort(arr, arr+sizee);
```

//Count Digits In A Number

```
log10(num)+1
```

//GCD

```
cout << "GCD: " << __gcd(n,m) << endl;
```

///To check if a value exists in a set

```
if (mySet.find(searchValue) != mySet.end()) {
```

```
    cout << searchValue << " is in the set." << endl;
```

///To get how many bits(1) in the binary form of n

```
store = __builtin_popcount(n)
```

///Sort in descending order

```
sort( num, num + n, greater<int>() );
```

///GCD(Greatest Common Divisor) of all number in a vector

```
vector<int> numbers = { 12, 15, 18, 21, 24 };
```

```
    int ans = __gcd(numbers[0], numbers[1]);
```

```
    for (int i = 2; i < numbers.size(); i++)
```

```
    {
```

```
        ans = __gcd(ans, numbers[i]);
```

```
    }
```

```
    cout << "The GCD of the numbers in the vector is: " << ans << endl;
```

//Decimal To Binary

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
string binary( long long int num )
```

```
{
    string get, add;
    long long int current;

    if( num > 1 )
        get = binary( num >> 1 );

    current = num & 1;

    add = get.append(to_string(current));
    return add;
}
```

```
int main()
{
    while( true )
    {
        cout << "Enter A Number: ";
        long long int num;
        cin >> num;

        long long int store_i;
        string store_s;

        store_s = binary(num);
        store_i = stoi(store_s);

        cout << store_s << endl;
        cout << store_i << endl;
    }
}
```

// In a Graph

Points or Axis (x_1, y_1) , (x_2, y_2) , (x_3, y_3) are on the same line, if $(x_2 - x_1)(y_3 - y_1) = (x_3 - x_1)(y_2 - y_1)$.

///Calculate prefix sum for operations like, arr[5] = {4,2,6,2,6}sum of index 2 to index 4 in O(1) complexity

calculating the prefix sum of the sequence beforehand. That is, let $sv_i = \sum_{j=1}^i v_j$. The sum of numbers in the interval $[l, r]$ would then be $sv_r - sv_{l-1}$

Connected Components In A Graph

```
#include <bits/stdc++.h>
using namespace std;
```

```
int connected_components( vector<vector<int>>&graph, int vertices )
{
    int total = 0, unvisited = vertices;

    unordered_map<int,int>visited;

    while( unvisited != 0 )
    {
        total++;
        int start;
        for( int i = 1; i <= vertices; i++ )
        {
            if( visited[i] != 1 )
            {
                start = i;
                break;
            }
        }
        stack<int>Stack;
        Stack.push(start);
        visited[start] = 1;
        unvisited--;
        while( !Stack.empty() )
        {
            int vertex = Stack.top();
            Stack.pop();
            for( int adjacent : graph[vertex] )
            {
                if( visited[adjacent] != 1 )
                {
                    Stack.push(adjacent);
                    visited[adjacent] = 1;
                    unvisited--;
                }
            }
        }
    }
    return total;
}

int main()
{
    int nodes, edges;
    cin >> nodes >> edges;
    vector<vector<int>>>Graph(nodes+1);
    for( int i = 0; i < edges; i++ )
    {
```

```
    int vertex1, vertex2;
    cin >> vertex1 >> vertex2;
    Graph[vertex1].push_back(vertex2);
    Graph[vertex2].push_back(vertex1);
}
cout << connected_components(Graph, nodes) << endl;
}
```

Minimum Spanning Tree Cost

```
int MST_Cost( vector<vector<pair<int,int>>>&graph, int vertices, int start )
{
    vector<int>visited(vertices,0);
    int target_edge = vertices-1, found_edge = 0, cost = 0;
    priority_queue< pair<int,pair<int,int>>, vector<pair<int,pair<int,int>>>,
greater<pair<int,pair<int,int>>> >Queue;
    for( auto point : graph[start] )
    {
        Queue.push(make_pair(point.second,make_pair(start,point.first)));
    }
    visited[start] = 1;
    while( !Queue.empty() && found_edge != target_edge )
    {
        pair<int,pair<int,int>>current;
        current = Queue.top();
        Queue.pop();
        int vertex1, vertex2, weight;
        weight = current.first;
        vertex1 = current.second.first;
        vertex2 = current.second.second;
        if( visited[vertex1] == 1 && visited[vertex2] == 1 )
        {
            continue;
        }
        cost += weight;
        int New;
        if( visited[vertex1] != 1 )
        {
            New = vertex1;
        }
        else
        {
            New = vertex2;
        }
        visited[New] = 1;
        for( auto point : graph[New] )
        {
            if( visited[point.first] != 1 )
            {
```

```
        Queue.push(make_pair(point.second,make_pair(New,point.first)));
    }
}
}
return cost;
}

    parent[point] = node;
}
else if( color[point] == 1 )
{
    int ancestor = parent[point], current = node;
    vector<int>store;
    while( current != ancestor )
    {
        store.push_back(current);

        current = parent[current];
    }
    store.push_back(ancestor);
    store.push_back(point);
    cycle.push_back(store);
}
}
}
}
}
}
int main()
{
    int n, m;
    cin >> n >> m;
    vector<vector<int>>>Graph(n+1), cycle;
    for( int i = 0; i < m; i++ )
    {
        int vertex1, vertex2;
        cin >> vertex1 >> vertex2;
        Graph[vertex1].push_back(vertex2);
        Graph[vertex2].push_back(vertex1);
    }
    Detect_Cycle(Graph,cycle,n);
    int total_cycle = cycle.size();

    cout << "Total Cycle: " << total_cycle << endl;
    for( int i = 0; i < total_cycle; i++ )
    {
        cout << "Cycle " << i+1 << ": ";
        for( auto point : cycle[i] )
```

```
    {  
        cout << point << " ";  
    }  
    cout << endl;  
}  
}
```

Detect all cycles from all connected components in a graph

```
#include <bits/stdc++.h>  
using namespace std;  
void Detect_Cycle( vector<vector<int>>&graph, vector<vector<int>>&cycles, vector<int>&found,  
vector<int>&color, vector<int>&Parent, int vertex, int parent )  
{  
    found[vertex] = 1;  
    if ( color[vertex] == 2 )  
    {  
        return;  
    }  
    if ( color[vertex] == 1 )  
    {  
        vector<int>store;  
        int current = parent;  
        store.push_back(current);  
        while ( current != vertex )  
        {  
            current = Parent[current];  
            store.push_back(current);  
        }  
        cycles.push_back(store);  
        return;  
    }  
    Parent[vertex] = parent;  
    color[vertex] = 1;  
    for ( auto point : graph[vertex])  
    {  
        if ( point == Parent[vertex] )  
        {  
            continue;  
        }  
        Detect_Cycle(graph,cycles,found,color,Parent,point,vertex);  
    }  
    color[vertex] = 2;  
}  
int main()  
{  
    int n, m;
```

```
cin >> n >> m;
vector<vector<int>>>Graph(n+1), Cycles;
for( int i = 0; i < m; i++ )
{
    int vertex1, vertex2;
    cin >> vertex1 >> vertex2;
    Graph[vertex1].push_back(vertex2);
    Graph[vertex2].push_back(vertex1);
}
vector<int>color(n+1), Parent(n+1), Found(n+1);
for( int i = 1; i <= n; i++ )
{
    if( Found[i] != 1 )
    {
        Detect_Cycle(Graph,Cycles,Found,color,Parent,i,i);
    }
}
int total_cycle = Cycles.size();
cout << "Total_Cycle: " << total_cycle << endl;
for( int i = 0; i < total_cycle; i++ )
{
    cout << "Cycle " << i+1 << ": ";
    for( auto point : Cycles[i] )
    {
        cout << point << " ";
    }
    cout << endl;
}
}
```

Fast input output

```
ios_base::sync_with_stdio(false);
cin.tie(NULL);
```

ULLONG_MAX (Maximum value for a variable of type unsigned long long (All positive))

LLONG_MIN

LLONG_MAX

For sorting vector<pair<int,int>>Pair;

sort(vect.begin(), vect.end(), sortbysec);

sort(Pair.rbegin(),Pair.rend());

//Counting childs of every parent node

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int Recursive_DFS( vector<vector<int>>&tree, vector<int>&hashmap, int root )
```

```
{
    int leaf = 0;
    for( auto point : tree[root] )
```

```
{
    leaf += Recursive_DFS(tree,hashmap,point);
    leaf++;
}
hashmap[root] = leaf;
return leaf;
}
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n;
    cin >> n;
    vector<vector<int>>>Tree(n+1);
    vector<int>Hashmap(n+1);
    for( int i = 2; i <= n; i++ )
    {
        int value;
        cin >> value;
        Tree[value].push_back(i);
    }
    Recursive_DFS(Tree,Hashmap,1);
    for( int i = 1; i <= n; i++ )
    {
        cout << Hashmap[i] << " ";
    }
    cout << "\n";
}
```

//Dijkstra with priority queue

```
#include <bits/stdc++.h>
using namespace std;
#define endl "\n"
#define ll unsigned long long int
void Dijkstra( vector<vector<pair<int,int>>>&graph, vector<pair<int,int>>&table, int start, int
vertices )
{
    for( int i = 1; i <= vertices; i++ )
    {
        if( i == start )
        {
            table[i] = make_pair(0,i);
        }
        else
        {

```

```
        table[i] = make_pair(INT_MAX, NULL);
    }
}
int total_visited = 0;
vector<int> visited(vertices+1);
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> Queue;
Queue.push(make_pair(table[start].first, start));
while( !Queue.empty() )
{
    pair<int,int> current = Queue.top();
    Queue.pop();
    int vertex, distance;
    vertex = current.second;
    distance = current.first;
    if( visited[vertex] == 1 )
    {
        continue;
    }
    visited[vertex] = 1;
    total_visited++;
    for( auto point : graph[vertex] )
    {
        int node, weight, previous_distance, new_distance;
        node = point.first;
        if( visited[node] == 1 )
        {
            continue;
        }
        weight = point.second;
        previous_distance = table[node].first;
        new_distance = table[vertex].first + weight;
        if( new_distance < previous_distance )
        {
            table[node].first = new_distance;
            table[node].second = vertex;
            Queue.push(make_pair(new_distance, node));
        }
    }
}
}
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int vertex, edge;
    cin >> vertex >> edge;
    vector<vector<pair<int,int>>> Graph(vertex+1);
```

```
for( int i = 0; i < edge; i++ )
{
    int v1, v2, weight;
    cin >> v1 >> v2 >> weight;
    Graph[v1].push_back(make_pair(v2,weight));
    Graph[v2].push_back(make_pair(v1,weight));
}
vector<pair<int,int>>Table(vertex+1);
Dijkstra(Graph,Table,1,vertex);
if( Table[vertex].first == INT_MAX )
{
    cout << -1 << endl;
}
else
{
    int current = vertex;
    vector<int>Path;
    Path.push_back(current);
    while( current != 1 )
    {
        current = Table[current].second;
        Path.push_back(current);
    }
    for( auto itr = Path.rbegin(); itr != Path.rend(); itr++ )
    {
        cout << *itr << " ";
    }
}
}
```

Formula

$\text{Sum}_{\text{even}} = n(n+1)$
 $\text{Sum}_{\text{odd}} = n^2$
 $\text{Sum all } n \text{ numbers} = n(n+1)/2$

//Floyd Warshall Algorithm

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long int
void Floyd( vector<vector<ll>>&graph, ll vertices )
{
    for( ll k = 1; k <= vertices; k++ )
    {
        for( ll i = 1; i <= vertices; i++ )
        {
            for( ll j = 1; j <= vertices; j++ )
            {
```

```
        if( ( graph[i][k] + graph[k][j] < graph[i][j] ) && graph[i][k] != LLONG_MAX && graph[k][j] !=
LLONG_MAX )
        {
            graph[i][j] = graph[i][k] + graph[k][j];
        }
    }
}
}
}
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    ll n, m, q;
    cin >> n >> m >> q;
    vector<ll>Length(n+1,LLONG_MAX);
    vector<vector<ll>>>Graph(n+1,Length), T_Graph;
    for( int i = 1; i <= n; i++ )
    {
        Graph[i][i] = 0;
    }
    for( ll i = 0; i < m; i++ )
    {
        ll vertex1, vertex2, weight;
        cin >> vertex1 >> vertex2 >> weight;
        if( Graph[vertex1][vertex2] > weight )
        {
            Graph[vertex1][vertex2] = weight;
            Graph[vertex2][vertex1] = weight;
        }
    }
    T_Graph = Graph;
    Floyd(T_Graph,n);
    for( int i = 0; i < q; i++ )
    {
        int vertex1, vertex2;
        cin >> vertex1 >> vertex2;
        if( T_Graph[vertex1][vertex2] != LLONG_MAX )
        {
            cout << T_Graph[vertex1][vertex2] << "\n";
        }
        else
        {
            cout << -1 << "\n";
        }
    }
}
```

```
ifstream Input("input.txt");
ofstream Output("output.txt");
ll row, column;
Input >> row >> column;
Output << X << ' ' << Y << "\n";

//Bellman Ford Single Source Shortest Path With Negative Cycle Detection
#include <bits/stdc++.h>
using namespace std;
bool Bellman_Ford( vector<vector<pair<int,int>>>&graph, vector<int>&distance, int vertices, int
start )
{
    bool negative_cycle = false;
    distance[start] = 0;
    for( int i = 1; i <= vertices-1; i++ )
    {
        for( int j = 1; j <= vertices; j++ )
        {
            for( auto point : graph[j] )
            {
                int from, to, weight;
                from = j;
                to = point.second;
                weight = point.first;
                if( distance[from]+weight < distance[to] )
                {
                    distance[to] = distance[from]+weight;
                }
            }
        }
    }
    for( int j = 1; j <= vertices; j++ )
    {
        for( auto point : graph[j] )
        {
            int from, to, weight;
            from = j;
            to = point.second;
            weight = point.first;
            if( distance[from]+weight < distance[to] )
            {
                negative_cycle = true;
                distance[to] = distance[from]+weight;
            }
        }
    }
    return negative_cycle;
}
```

```
}
int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    int n, m;
    cin >> n >> m;
    vector<vector<pair<int,int>>>Graph(n+1);
    for( int i = 0; i < m; i++ )
    {
        int vertex1, vertex2, weight;
        cin >> vertex1 >> vertex2 >> weight;
        Graph[vertex1].push_back(make_pair(weight,vertex2));
    }
    vector<int>Distance(n+1,INT_MAX);
    int start = 1;
    bool negative_cycle = Bellman_Ford(Graph,Distance,n,start);
    if( negative_cycle == true )
    {
        cout << "Cycle Detected" << endl;
    }
    else
    {
        for( int i = 1; i <= n; i++ )
        {
            cout << i << ": " << Distance[i] << endl;
        }
    }
}
//Modulo
ll MOD = 1000000007;
ll Modulo(ll base, ll exponent)
{
    if (exponent == 0)
    {
        return 1;
    }
    if (exponent == 1)
    {
        return base % MOD;
    }
    ll temp = Modulo(base, exponent / 2);
    if (exponent % 2 == 0)
    {
        return (temp * temp) % MOD;
    }
    else
    {

```

```
        return (((temp * temp) % MOD) * base) % MOD;
    }
}

//Total number of divisors of all numbers from 1 to 1000001
void Func( vector<int>&divisors, int n )
{
    for( int i = 1; i <= n; i++ )
    {
        for( int j = i; j <= n; j += i )
        {
            divisors[j]++;
        }
    }
}

//Difference Array
#include <bits/stdc++.h>
using namespace std;
#define endl "\n"
#define printDA for( auto point : DA ){cout << point << " " << endl;}
void Difference_Array( vector<int>&DA, vector<int>arr )
{
    int length = arr.size();
    DA.push_back(arr[0]);
    for( int i = 1; i < length; i++ )
    {
        DA.push_back(arr[i]-arr[i-1]);
    }
}

void Update_DA( vector<int>&DA, int l, int r, int value )
{
    DA[l] += value;
    if( r < DA.size()-1 )
    {
        DA[r+1] -= value;
    }
}

void Print_DA( vector<int>&DA, vector<int>&A )
{
    int length = DA.size();
    cout << "Print " << length << endl;
    for( int i = 0; i < length; i++ )
    {
        if( i == 0 )
        {
            A.push_back(DA[i]);
        }
    }
}
```

```
        else
        {
            A.push_back(DA[i]+A[i-1]);
        }
        cout << A[i] << " ";
    }
}
int main()
{
    int length;
    cin >> length;
    vector<int>store;
    for( int i = 0; i < length; i++ )
    {
        int value;
        cin >> value;
        store.push_back(value);
    }
    vector<int>DA;
    Difference_Array(DA,store);
    printDA;
    Update_DA(DA,2,4,10);
    vector<int>Array;
    Print_DA(DA,Array);
}
```

//DSU

```
class DSU{
private:
    int Total_Union;
    vector<int>Parent, Union_Size;
public:
    DSU( int vertices )
    {
        Total_Union = vertices;
        for( int i = 0; i <= vertices; i++ )
        {
            Parent.push_back(i);
            Union_Size.push_back(1);
        }
    }
    void Union( int vertex1, int vertex2 )
    {
        int union1, union2;
        union1 = Find_Union(vertex1);
```

```
union2 = Find_Union(vertex2);
if( union1 != union2 )
{
    int first, second;
    if( Union_Size[union1] >= Union_Size[union2] )
    {
        first = union1;
        second = union2;
    }
    else
    {
        first = union2;
        second = union1;
    }
    Parent[second] = first;
    Union_Size[first] += Union_Size[second];
    Union_Size[second] = 0;
    Total_Union--;
}
}
int Find_Union( int vertex )
{
    int current = vertex;
    while( current != Parent[current] )
    {
        current = Parent[current];
    }
    int root = current;
    while( Parent[vertex] != root )
    {
        int temp = Parent[vertex];
        Parent[vertex] = root;
        vertex = temp;
    }
    return root;
}
int Unions()
{
    return Total_Union;
}
};
```

The following function prime checks if the given number n is prime. $O(\sqrt{n})$ complexity

```
bool prime(int n) {  
    if (n < 2) return false;  
    for (int x = 2; x*x <= n; x++) {  
        if (n%x == 0) return false;  
    }  
    return true;  
}
```

Prime factors:

```
vector<int> factors(int n) {  
    vector<int> f;  
    for (int x = 2; x*x <= n; x++) {  
        while (n%x == 0) {  
            f.push_back(x);  
            n /= x;  
        }  
    }  
    if (n > 1) f.push_back(n);  
    return f;  
}
```

The sieve of Eratosthenes is a preprocessing algorithm that builds an array using which we can efficiently check if a given number between 2...n is prime and, if it is not, find one prime factor of the number.

```
for (int x = 2; x <= n; x++) {  
    if (sieve[x]) continue;  
    for (int u = 2*x; u <= n; u += x) {  
        sieve[u] = x;  
    }  
}
```

Euler's totient function

Numbers a and b are **coprime** if $\gcd(a, b) = 1$. **Euler's totient function** $\varphi(n)$ gives the number of coprime numbers to n between 1 and n . For example, $\varphi(12) = 4$, because 1, 5, 7 and 11 are coprime to 12.

The value of $\varphi(n)$ can be calculated from the prime factorization of n using the formula

$$\varphi(n) = \prod_{i=1}^k p_i^{\alpha_i-1} (p_i - 1).$$

For example, $\varphi(12) = 2^1 \cdot (2 - 1) \cdot 3^0 \cdot (3 - 1) = 4$. Note that $\varphi(n) = n - 1$ if n is prime.

Modular arithmetic

Often we can take remainders before doing calculations. In particular, the following formulas hold:

$$(x + y) \bmod m = (x \bmod m + y \bmod m) \bmod m$$

$$(x - y) \bmod m = (x \bmod m - y \bmod m) \bmod m$$

$$(x \cdot y) \bmod m = (x \bmod m \cdot y \bmod m) \bmod m$$

$$x^n \bmod m = (x \bmod m)^n \bmod m$$

1. $(a+b)\%c = ((a\%c)+(b\%c))\%c$
2. $(a*b)\%c = ((a\%c)*(b\%c))\%c$
3. $(a-b)\%c = ((a\%c)-(b\%c)+c)\%c$
4. $(a/b)\%c = ((a\%c)/(b\%c))\%c$

//MOD Inverse

```
int modInverse(int a, int m)
{
    int m0 = m;
    int y = 0, x = 1;

    if (m == 1)
        return 0;

    while (a > 1)
    {
        // q is quotient
        int q = a / m;
        int t = m;

        // m is remainder now, process same as
        // Euclid's algo
        m = a % m, a = t;
        t = y;

        // Update y and x
        y = x - q * y;
        x = t;
    }

    // Make x positive
    if (x < 0)
        x += m0;

    return x;
}
```

// C++ program to find large

// factorials using BigInteger

```
#include <bits/stdc++.h>
using namespace std;

#define ull unsigned long long

// Returns Factorial of N
ull factorial(int N)
{
    // Initialize result
    ull f = 1; // Or BigInt 1

    // Multiply f with 2, 3, ...N
    for (ull i = 2; i <= N; i++)
        f *= i;
    return f;
}

// Driver method
int main()
{
    int N = 20;
    cout << factorial(N) << endl;
}
```

0 1 knapsack dp

```
int knapSack(int W, vector<int>& wt, vector<int>& val) {
    int n = wt.size();
    vector<vector<int>> dp(n + 1, vector<int>(W + 1));

    // Build table dp[][] in bottom-up manner
    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]],
                               dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }
    return dp[n][W];
}
```

Arithmetic Operations

Sum of the first n natural numbers:

$$S = \frac{n \times (n + 1)}{2}$$

Sum of the squares of the first n natural numbers:

$$S = \frac{n \times (n + 1) \times (2n + 1)}{6}$$

Sum of the cubes of the first n natural numbers:

$$S = \left(\frac{n \times (n + 1)}{2} \right)^2$$

Quadratic Equation

Roots of the quadratic equation $ax^2 + bx + c = 0$:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Used to find the solutions of a quadratic equation.

Distance Between Two Points

Distance between (x_1, y_1) and (x_2, y_2) :

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Used in geometry problems to calculate the Euclidean distance.

. Area of a Triangle (Given Vertices)

- Area of a triangle with vertices (x_1, y_1) , (x_2, y_2) , and (x_3, y_3) :

$$\text{Area} = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|$$

Used to calculate the area of a triangle when the coordinates of its vertices are known.

Area of a circle with radius r :

$$A = \pi r^2$$

Used to find the area enclosed by a circle.

Circumference of a circle with radius r :

$$C = 2\pi r$$

This formula gives the perimeter or boundary length of a circle.

Volume of a cuboid with length l , width w , and height h :

$$V = l \times w \times h$$

Volume of a sphere with radius r :

$$V = \frac{4}{3}\pi r^3$$

Volume of a Cone

Volume of a cone with radius r and height h :

$$V = \frac{1}{3}\pi r^2 h$$

Volume of a Cylinder

Volume of a cylinder with radius r and height h :

$$V = \pi r^2 h$$

//Concept of the distance of the furthest node from any node

```
void Depth_Node( vector<vector<int>>&tree, vector<int>&level, int &Max_Depth, int
&Deepest_Node, int vertices, int start )
{
    vector<int>visited(vertices+1);
    stack<int>Stack;
    Stack.push(start);
    visited[start] = 1;
    level[start] = 0;
    while( !Stack.empty() )
    {
        int node = Stack.top();
        Stack.pop();
        if( level[node] >= Max_Depth )
        {
            Max_Depth = level[node];
        }
    }
}
```

```
        Deepest_Node = node;
    }
    for( auto point : tree[node] )
    {
        if( visited[point] != 1 )
        {
            visited[point] = 1;
            Stack.push(point);
            level[point] = level[node]+1;
        }
    }
}
}
int main()
{
    int n, start = 0;
    cin >> n;
    vector<vector<int>>>Tree(n+1);
    for( int i = 0; i < n-1; i++ )
    {
        int vertex1, vertex2;
        cin >> vertex1 >> vertex2;
        if( start == 0 )
        {
            start = vertex1;
        }
        Tree[vertex1].push_back(vertex2);
        Tree[vertex2].push_back(vertex1);
    }
    int Max_Depth = 0, Deepest_Node;
    vector<int>Level_One(n+1,0), Level_Two(n+1,0);
    Depth_Node(Tree,Level_One,Max_Depth,Deepest_Node,n,start);
    int first = Deepest_Node;
    Depth_Node(Tree,Level_Two,Max_Depth,Deepest_Node,n,first);
    int second = Deepest_Node;
    vector<int>first_distance(n+1), second_distance(n+1);
    Depth_Node(Tree,first_distance,Max_Depth,Deepest_Node,n,first);
    Depth_Node(Tree,second_distance,Max_Depth,Deepest_Node,n,second);
    for( int i = 1; i <= n; i++ )
    {
        cout << max(first_distance[i],second_distance[i]) << " ";
    }
    cout << "\n";
}
```